Git&GitHub 가이드

저장소 생성·선택부터 Release / Tag / CI/CD까지 한 번에 복습하고 실습하기

학습 방식	GitHub Desktop + GitHub 웹, 필요한 경우에만 Git 명령어
실습 대상	예제 저장소 1개와 index.html 파일 1개
완료 기준	변경 → 검토 → 병합 → 자동 배포 → 버전 발행 확인

이 교재는 Git/GitHub의 실무 기본 기능을 빠르게 복습하기 위한 자료다. 개념 설명은 짧게, 조작 순서와 확인 결과는 구체적으로 다룬다.

전체 흐름

Repository → Commit → Push/Pull → Branch → Pull Request → Merge → Revert → Actions(CI/CD) → Tag → Release

메뉴 명칭은 GitHub Desktop 버전 및 저장소 상태에 따라 조금씩 달라질 수 있다. 단계를 끝낼 때마다 GitHub 웹에서 결과를 확인한다.

UI 이미지 보강 · 2026-10-13

공식 GitHub Docs 화면을 각 실습 페이지에 추가했다. 원문 보기는 클릭할 수 있다.

이미지는 기능 확인용 예시이며, 운영체제·버전에 따라 메뉴 표시가 다를 수 있다.

01. 기본 개념 빠르게 복습

기능	무엇을 하는가
Repository	파일과 Git 이력을 관리하는 저장소
Clone	원격 저장소를 로컬 PC로 복제
Commit	선택한 변경사항을 로컬 이력에 기록
Push	로컬 커밋을 원격 GitHub에 전송
Fetch / Pull	원격 변경 확인 / 원격 변경을 가져와 통합
Branch	독립된 변경 작업 흐름
Pull Request	변경 검토 및 병합 요청
Merge	브랜치의 변경 내용을 다른 브랜치에 통합
Tag / Release	버전 지점 표시 / 배포 버전 설명·파일 게시
CI / CD	변경 검사 자동화 / 배포 자동화

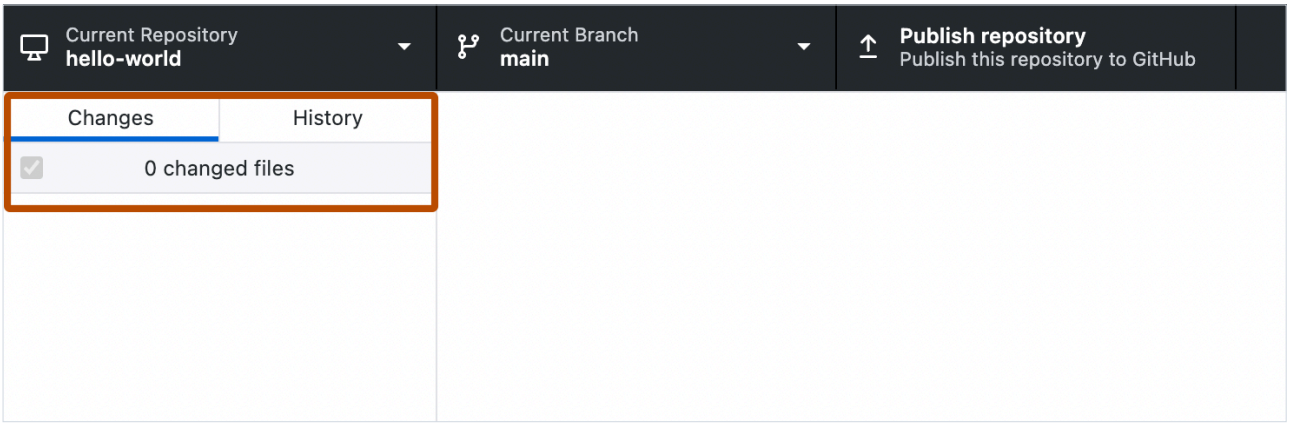
구분할 것

파일 저장(Ctrl+S) ≠ Commit ≠ Push. Commit은 변경 기록이고 Push가 원격 전송이다.

GitHub Desktop의 주요 영역

Changes: 미커밋 변경 / History: 커밋 기록 / Summary: 커밋 제목 / Description: 상세 설명 / Current branch: 작업 브랜치 / Fetch·Pull·Push origin: 원격 동기화

화면 대조 | GitHub Desktop 주요 영역



상단: Current repository / Current branch / 원격 동기화

왼쪽: Changes / History. 화면 속 저장소 이름은 공식 문서의 예시다.

공식 문서 · 원문 보기

02. 저장소 생성 또는 기존 저장소 선택

A. 새 저장소 만들기

- GitHub Desktop → File → New repository.
- Name에 git-guide-practice 입력, Local path 선택 → Create repository.
- Repository → Show in Explorer로 실제 로컬 폴더 확인.
- Publish repository → GitHub에 게시. 처음에는 Private로 설정해도 된다.

B. 이미 GitHub에 만든 저장소 가져오기

- GitHub Desktop → File → Clone repository → GitHub.com 탭.
- 대상 저장소 선택 → Local path 지정 → Clone.
- Current repository에서 지금 선택한 저장소 이름 확인.

C. 기존 로컬 폴더를 Git에 연결하기

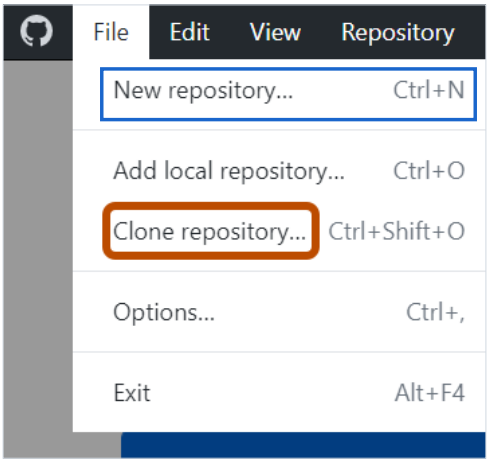
이미 Git 저장소인 폴더라면 File → Add local repository에서 선택한다. Git 저장소가 아니라면 새 저장소 생성 또는 해당 폴더를 Git 저장소로 초기화해야 한다.

결과 확인

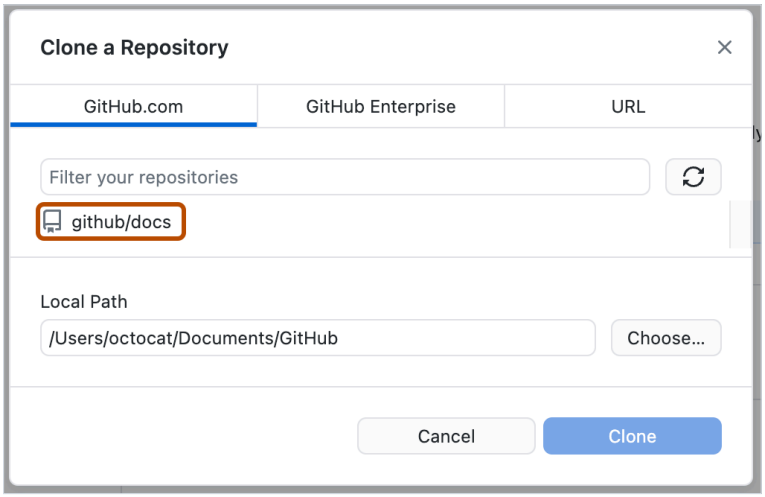
① 로컬 폴더 존재 ② Desktop에서 저장소 선택 가능 ③ Publish/Clone한 경우 GitHub 웹의 저장소와 연결 확인.

화면 대조 | A. 생성 메뉴 · B. Clone 화면

① File → New repository (파란 테두리)



② GitHub.com → 저장소 → Local Path → Clone

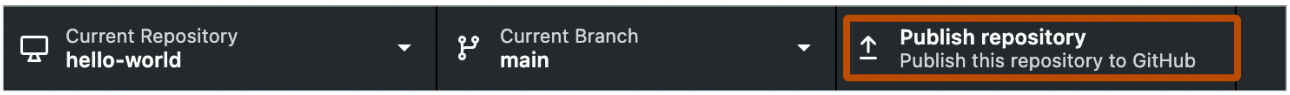


왼쪽은 Windows 메뉴, 오른쪽은 공식 문서의 macOS 화면이다.

오른쪽 경로는 예시이며, Windows에서는 본인의 로컬 폴더를 지정한다.

공식 문서 · 원문 보기

③ 새 저장소를 만들었다면 Publish repository로 게시



공식 문서 · 원문 보기

03. 파일 변경 → Commit → Push → Pull

첫 커밋

- 저장소 폴더에 index.html 생성. 아래 예시를 저장한다.
- GitHub Desktop → Changes에서 index.html 선택 및 변경 미리보기.
- Summary: docs: 첫 실습 파일 추가 / Description: 기본 페이지 구성.
- Commit to main 클릭 → History에 새 커밋 확인.
- Push origin 클릭 → GitHub 웹에서 index.html 확인.

```
<!doctype html>
<html lang="ko">
<head><meta charset="utf-8"><title>Git Practice</title></head>
<body><h1>Version 1</h1></body>
</html>
```

두 번째 커밋

- Version 1을 Version 2로 수정하고 저장.
- Changes에서 차이 비교 → Summary: docs: 페이지 문구 수정.
- Commit → Push → 웹에서 최신 변경 확인.

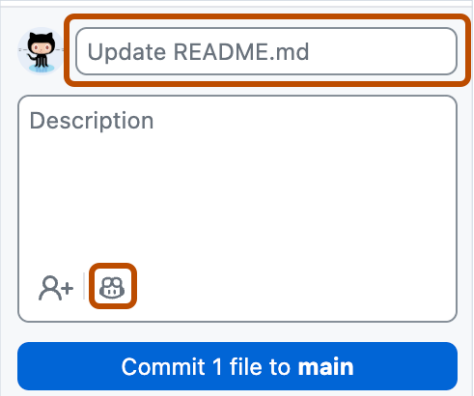
Pull 연습

- GitHub 웹에서 README.md를 편집 후 Commit changes.
- Desktop에서 Fetch origin → Pull origin (필요 시) 실행.
- 로컬 README.md에 웹 변경 내용이 반영됐는지 확인.

주의: 같은 파일을 로컬/웹에서 동시에 수정하면 충돌이 날 수 있다. 먼저 Fetch/Pull로 최신 상태를 확인한다.

화면 대조 | Commit 입력란과 원격 동기화

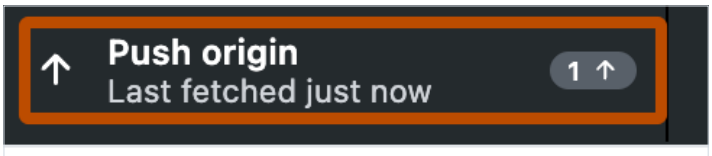
① Summary / Description / Commit



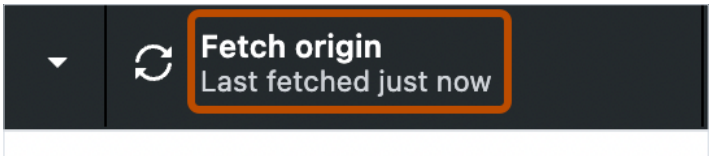
공식 문서 · 원문 보기

이미지의 예제 메시지 대신
본문의 Summary를 입력한다.

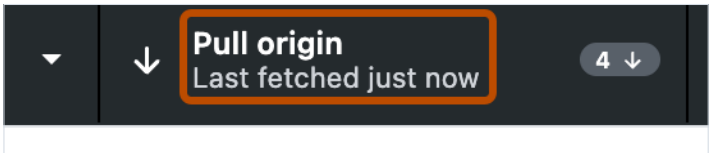
② Push origin: 로컬 커밋 전송



③ Fetch origin: 원격 변경 확인



④ Pull origin: 원격 변경 통합



공식 문서 · 원문 보기

04. Branch → Pull Request → Merge

별도 브랜치에서 변경하기

- Desktop → Current branch → New branch → feature/title-change.
- index.html의 제목을 Git Practice Updated로 변경.
- Changes 검토 → Commit to feature/title-change → Publish branch 또는 Push origin.

Pull Request 만들기

- Desktop → Branch → Create Pull Request 또는 GitHub 웹의 Compare & pull request.
- Base: main / Compare: feature/title-change 확인.
- 제목: docs: 페이지 제목 수정. 설명에는 변경 목적과 확인 결과 기록.
- Create pull request → Files changed에서 차이를 검토.

병합 후 정리

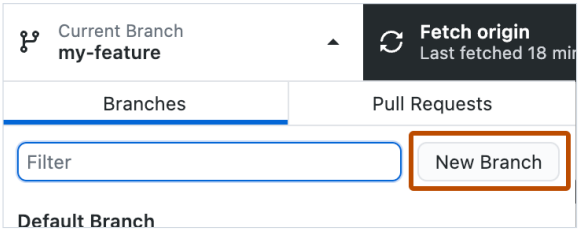
- GitHub 웹에서 Merge pull request → Confirm merge (병합 가능한 경우).
- Desktop에서 main 선택 → Fetch/Pull origin.
- main의 index.html에 변경이 들어왔는지 확인.
- 작업이 끝난 브랜치는 필요에 따라 삭제.

커밋 제목 실무형 예시

docs: 사용법 보완 / feat: 페이지 메뉴 추가 / fix: 잘못된 링크 수정 / refactor: 파일 구조 정리 / ci: 배포 워크플로 추가

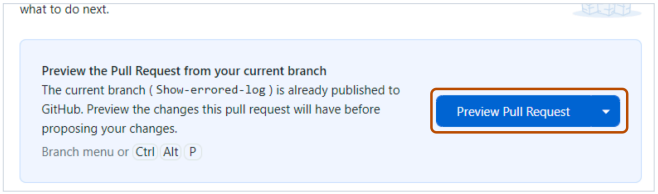
화면 대조 | 브랜치 생성 → PR 미리보기 → 병합

① Current branch → New Branch



공식 문서 · 원문 보기

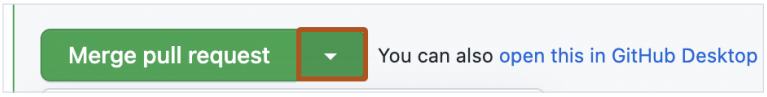
② Preview Pull Request (Windows)



미리보기 후 Create Pull Request로 진행.

공식 문서 · 원문 보기

③ 웹에서 Merge pull request



병합 방식 메뉴가 열린 공식 화면의 상단 부분.

이 교재에서는 본문과 같이 main을 병합 대상(base)으로 확인한다.

공식 문서 · 원문 보기

05. 변경 취소와 충돌의 기본

상황에 따른 선택

상황	일반적인 처리
아직 Commit 전	Changes에서 파일 변경 비교 후 필요 시 Discard changes
이미 Commit + Push 완료	History에서 해당 커밋을 Revert하여 취소 커밋 생성
원격에서 추가 변경 발생	Fetch/Pull 후 차이 확인, 충돌 발생 시 수동 수정

Revert 실습

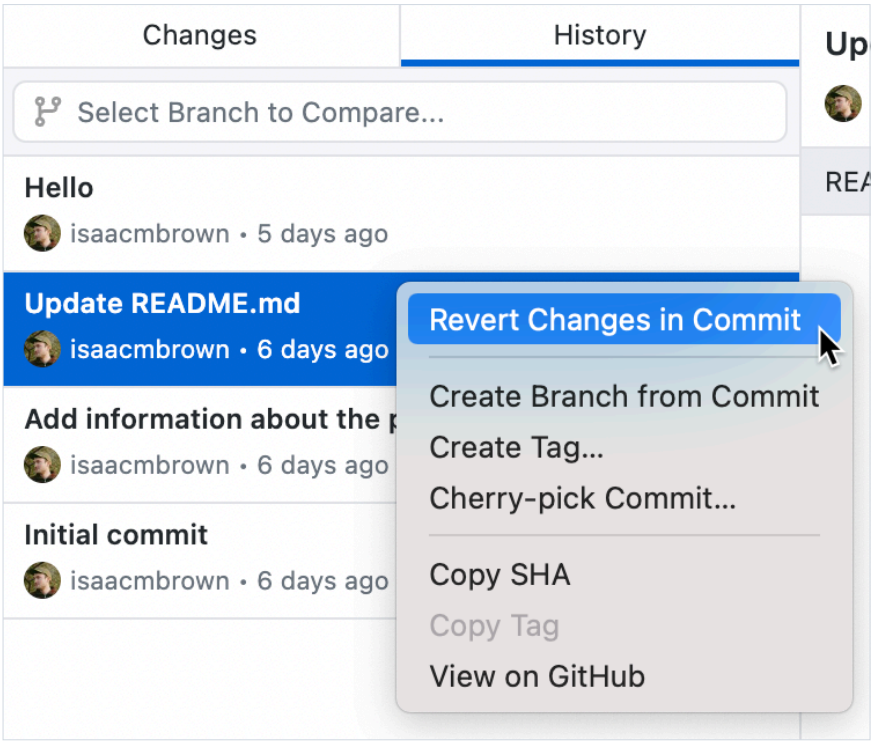
- main에서 문구를 한 번 변경하고 Commit → Push.
- Desktop → History → 해당 커밋 우클릭 → Revert this commit (표시되는 경우).
- 새로 생성된 취소 커밋을 확인하고 Push.
- GitHub 웹에서 원래 문구가 복원됐는지 확인.

충돌이 났다면

충돌 파일에서 어느 내용을 남길지 결정 → 충돌 표시 제거 → 파일 저장 → Desktop에서 해결 상태 확인 → Commit.
무조건 Force Push로 해결하려 하지 않는다.

참고: Revert는 이전 커밋 자체를 삭제하는 방식이 아니라 변경을 되돌리는 새 커밋을 남기는 방식이다.

화면 대조 | History에서 변경을 되돌리는 메뉴



공식 문서 화면(macOS)

History → 커밋 우클릭

Revert Changes in Commit 선택.

본문의 Revert와 같은 기능이다.

취소 커밋 생성 후 Push로 원격에 반영.

공식 문서 · 원문 보기

06. GitHub Actions: CI와 CD 맛보기

이 실습에서는 정적 HTML 파일을 사용한다. CI는 파일 존재 여부를 검사하고, CD는 GitHub Pages에 사이트를 배포한다.

GitHub 설정

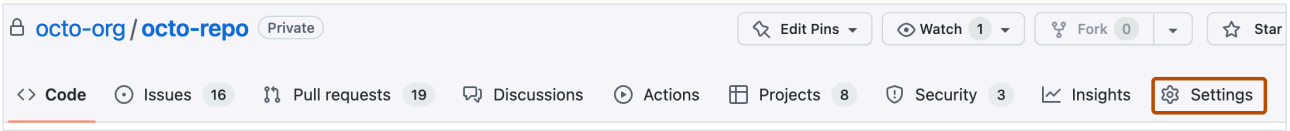
- GitHub 웹 → Settings → Pages → Build and deployment → Source: GitHub Actions.
- 저장소에 .github/workflows/deploy.yml 파일을 만든다. 아래 내용을 입력하고 Commit.
- Actions 탭에서 실행 결과 확인 → 성공하면 Pages의 게시 주소 열기.

```
name: Check and Deploy
on:
  push:
    branches: [main]
  workflow_dispatch:
permissions:
  contents: read
  pages: write
  id-token: write
jobs:
  deploy:
    runs-on: ubuntu-latest
    environment:
      name: github-pages
      url: ${{ steps.deployment.outputs.page_url }}
    steps:
      - uses: actions/checkout@v6
      - name: Check HTML (CI)
        run: test -f index.html
      - uses: actions/configure-pages@v5
      - uses: actions/upload-pages-artifact@v4
        with:
          path: .
      - id: deployment
        uses: actions/deploy-pages@v4
```

Public 저장소로 진행하는 것이 가장 간단하다. Private 저장소의 Pages 이용 가능 여부는 계정 플랜에 따라 다를 수 있다.

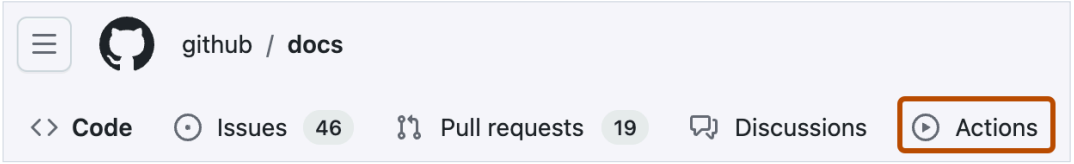
화면 대조 | Pages 설정 진입과 Actions 실행 확인

① 저장소의 Settings 탭 → 왼쪽 Pages → Source: GitHub Actions



공식 문서 · 원문 보기

② 저장소의 Actions 탭에서 Check and Deploy 실행 선택



공식 문서 · 원문 보기

07. CI/CD 재실행과 오류 확인

검증 → 배포 결과 확인

- main의 index.html 문구 수정 → Commit → Push.
- GitHub → Actions → 최신 Check and Deploy 실행 선택.
- Check HTML 단계 성공과 Deploy 단계 성공을 각각 확인.
- Pages 주소를 새로고침해 수정된 문구 확인.

일부러 실패시키기

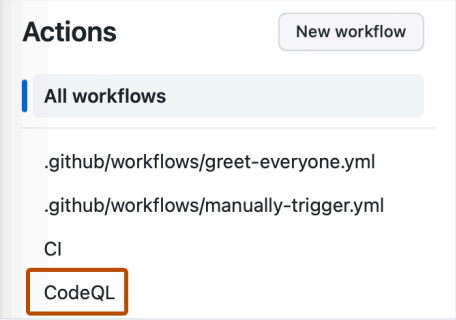
- index.html 이름을 temp.html로 변경 → Commit → Push.
- Actions에서 Check HTML 실패 확인.
- 파일 이름을 다시 index.html로 복구 → Commit → Push.
- Actions 성공 및 사이트 재배포 확인.

자주 확인할 문제

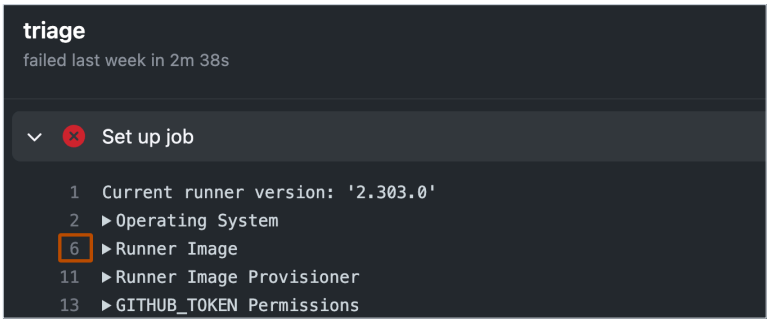
증상	확인
Actions가 실행되지 않음	워크플로 경로, main push 여부, Actions 설정
CI 검사 실패	index.html 파일 유무와 경로
배포 실패	Settings → Pages의 Source, Actions의 오류 로그, 권한
사이트가 업데이트되지 않음	최근 워크플로 성공 여부와 실제 Pages URL

화면 대조 | 워크플로 선택과 실패 단계의 로그

① Actions 왼쪽 목록



② 실행 → Job → 실패한 단계 펼치기



왼쪽 CodeQL과 오른쪽 Set up job은 공식 문서의 예시다.

실습에서는 Check and Deploy → deploy → Check HTML을 확인한다.

빨간 표시의 단계를 펼쳐 오류를 읽고, 파일명 복구 후 새 실행을 확인한다.

공식 문서 · 실행 기록

공식 문서 · 실행 로그

08. Tag와 Release: 한 사이클 마무리

태그와 릴리스 구분

Tag는 특정 커밋에 붙이는 버전 이름(예: v1.0.0)이고, Release는 해당 버전을 설명하고 배포 파일을 첨부할 수 있는 GitHub 게시 기능이다.

Release 생성 실습

- GitHub 웹 → 저장소 → Releases → Draft a new release.
- Choose a tag → v1.0.0 입력 → Create new tag 선택.
- Target이 배포하려는 main의 최신 커밋인지 확인.
- Release title: v1.0.0 / 설명: 첫 실습 버전, Pages 배포 완료.
- Publish release → Releases 화면의 태그·설명 확인.

마지막 점검

- GitHub Desktop에서 저장소 선택·Commit·Push·Pull 완료.
- feature 브랜치 생성 → PR → Merge 완료.
- Revert 또는 충돌 처리의 개념 설명 가능.
- Actions 검사 및 GitHub Pages 자동 배포 성공.
- v1.0.0 Tag와 Release 게시 확인.

참고 자료

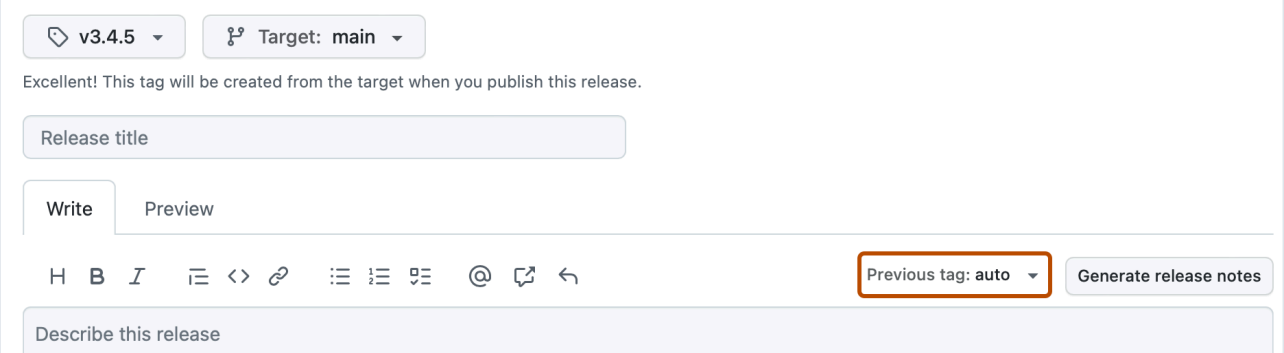
GitHub Desktop: https://docs.github.com/ko/desktop

GitHub Pages 자동 배포: https://docs.github.com/ko/get-started/start-your-journey/deploying-your-website-automatically

GitHub Releases: https://docs.github.com/ko/repositories/releasing-projects-on-github

문서 안내: 예제는 GitHub 기능 실습을 위해 구성했으며 실제 프로젝트 요구사항을 전제하지 않는다.

화면 대조 | Release 작성 화면



화면의 버전은 공식 예시다. 실습에서는 v1.0.0 / Target: main을 확인한다.

공식 문서 · 원문 보기